

Assignment - 3

Ans to the q.no-01

Here is the computer program to identify the root for $y = 10x^5 - 9x^3 + 7x$.

```
#include <bits/stdc++.h>
```

```
using namespace std.
```

```
# define ld long double
```

```
ld f(ld x)
```

```
{ return  $10x^5 - 9x^3 + 7x$ ; }
```

```
ld df (ld x)
```

```
{ return  $50x^4 - 27x^2 + 7$ ; }
```

```
ld Error ( ld a, ld b)
```

```
{ return  $\text{abs}((a-b)/a)$ ; }
```

```
int iter=0
```

```
ld NewtonRaphson(ld a, ld erron)
```

```
{ ld b = a - f(a)/df(a);
```

```
while (Erron(a,b) > erron)
```

```
{ cout << fixed << setprecision(4) << iter << " /t"  
<< a << " /t" << b << " /t" << Erron(a,b) << endl;
```

```
a = b;
```

```
b = a - f(a)/df(a);
```

```
iter++;
```

```
}
```

```
cout << "The root of the equation is:" << endl;
```

```
return b;
```

```
}
```

```
int main()
```

```
{ ld a; cin >> a;
```

```
ld erron = 0.001;
```

```
cout << "Newton-Raphson Method" << endl;
```

```
cout << "Iter" << " /t" << a << " /t" << b << " /t" << Erron
```

```
: " << endl;
```

```
cout << NewtonRaphson(a, erron) << endl;
```

```
return 0; }
```

Ans to the q. no-02

Here is the computer program to identify the root for $y = 10x^5 - 9x^3 + 7x$ using false position method.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define ldx long double
```

```
ldx f(ldx)
```

```
{ return  $10x^5 - 9x^3 + 7x$  }
```

```
ldx Error (ldx a, ldx b)
```

```
{ return  $\text{abs}((a-b)/a)$ ; }
```

```
int iter = 0;
```

```
ldx Falseposition (ldx a, ldx b, ldx error)
```

```
{ ldx c = 0;
```

```
while (Error (a,b) > error)
```

```
{  $c = (a \cdot f(b) - b \cdot f(a)) / (f(b) - f(a))$ ;
```

```
cout << fixed << setprecision (4) << iter
```

```
<< " \t\t" << a << " \t\t" << b << " \t\t"
<< c << " \t\t" Error (a, b) << endl;
```

```
if (f(c) = 0)
```

```
{ return c; }
```

```
else if (f(c) * f(a) < 0)
```

```
    b = c;
```

```
else
```

```
    a = c;
```

```
    iter++;
```

```
}
```

```
cout << "The root of the equation is: " << endl;
```

```
return c;
```

```
}
```

```
int main ()
```

```
{    Id a = 1, b = 1;
```

```
    Id error = 0.01;
```

```
    cout << "False position method" << endl;
```

```
    cout << "Iter" << " \t\t a:" << " \t\t b:" << " \t\t c:"
```

```
    << " \t\t Error:" << endl;
```

```
    cout << Falseposition (a, b, error) << endl;
```

```
    return 0; }
```

Ans to the q: no-03

Here is the computer program for identify the root for $y = 10x^5 - 9x^3 + 7x$ using second method;

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
#define ld long double
```

```
ld f(ld x)
```

```
{ return  $10x^5 - 9x^3 + 7x$ ; }
```

```
ld Error (ld a, ld b)
```

```
{ return  $\text{abs}(a-b)/b$ ; }
```

```
int iter = 0;
```

```
ld secondMethod (ld x1, ld x2, ld error)
```

```
{ ld = c = 0;
```

```
while (Error (x1, x2) > error)
```

```
if ( $f(x_2) - f(x_1) = 0$ )
```

```
{ cout << "Division by Zero " iter << endl;
```

```
return c;
```

```
}
```

```

c = (x1 * f(x2) - x2 * f(x1)) / (f(x2) - f(x1));
cout << fixed << precision(4) << iter << "\t\t"
<< x1 << "\t\t" << x2 << "\t\t" << c << "\t\t"
Error(x1, x2) << endl;
if (f(c) == 0.0)
    break;
else { x1 = x2, x2 = c; }
iter++; }
cout << "The root of the equation is: " << endl;
return c;
}
int main()
{
    double x1 = 2.0, x2 = 1.0;
    double error = 0.0001;
    cout << "Secant Method" << endl;
    cout << "Iter" << "\t\t\t" << "x1" << "x2" << "f(x)"
    << "\t\t\tError" << endl;
    cout << secantMethod(x1, x2, error) << endl;
    return 0;
}

```

Ans to the q: no-04

Constructing the difference table:

θ	y	Δy	$\Delta^2 y$	$\Delta^3 y$
10	52			
15	25	-27		
20	12	-13	14	
25	10	-2	11	-3

Forward difference formula.

$$y(\theta) = y_0 + u \Delta y_0 + \frac{u(u-1)}{2!} \nabla^2 y_0 + \frac{u(u-1)(u-2)}{3!} \nabla^3 y_0$$

$$\text{For } \theta = 17 \quad u = \frac{\theta - \theta_0}{h} = \frac{17 - 10}{5} = 1.4$$

$$y(17) = 52 + 1.4(-27) + \frac{1.4 \times (0.4)}{2} \times 14 + \frac{1.4 \times 0.4 \times (-0.6)}{3!} \times (-3)$$

$$= 52 - 37.8 + 30.2 + 0.168$$

$$= 18.288$$

Backward Difference Formula:

$$y(\theta) = y_n + u \nabla y_n + \frac{u(u+1)}{2!} \nabla^2 y_n + \frac{u(u+1)(u+2)}{3!} \nabla^3 y_n$$

$$\text{Here, } u = \frac{17-25}{8} = -1.6$$

$$y(17) = 10 + (-1.6) \times (-2) + \frac{(-1.6) \times (-0.6) \times 11}{2} + \frac{(-1.6) \times (-0.6) \times 0.4 \times (-9)}{6}$$

$$= 10 + 3.2 + 5.28 - 0.192$$

$$= 18.288$$

After calculation we can see that both method are same value. It confirms that the interpolation was conducted using both forward and backward difference formulas.

Ans to the q. no-05

Derivation of the Trapezoidal rule:

The trapezoidal rule is used for approximating the definite integral of a function.

Given a continuous function $f(x)$ over the interval $[a, b]$, we can approximate the area under the curve by dividing it into n equally spaced subintervals.

1. Divide the interval:

→ Let $h = \frac{b-a}{n}$ be the width of each subinterval.

→ The points are $x_0 = a, x_1 = a+h, \dots, x_n = b$

2. Approximate the Area: The area under the curve can be approximated

as:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right)$$

Error Analysis:

The error in the trapezoidal rule given by

$$E_T = -\frac{(b-a)^3}{12n^2} f''(\xi)$$

Comparison with Simpson's 1/3 and 3/8 rules

1. Simpson's 1/3 Rule: For an even number of segments, it approximates the integral as

$$S_n = \frac{h}{3} \left(f(a) + 4 \sum_{i=1, \text{odd}}^{n-1} f(x_i) + 2 \sum_{i=2, \text{even}}^{n-2} f(x_i) + f(b) \right)$$

Error:

$$E_S = \frac{(b-a)^5}{80n^4} f^{(4)}(\xi)$$

2. Simpson's 3/8 Rule:

$$S_{3/8} = \frac{3h}{8} \left(f(a) + 3 \sum f(x_i) + 2 \sum f(x_i) + f(b) \right)$$

Error:

$$E_{3/8} = -\frac{(b-a)^5}{80n^4} f^{(4)}(\xi)$$

Comparison: $\rightarrow$ Trapezoidal Rule: $E_T = O\left(\frac{1}{n^2}\right)$

$\rightarrow$ Simpson's Rules $E_S, E_{3/8} = O\left(\frac{1}{n^4}\right)$

Simpson's Rules are more accurate than Trapezoidal rule.